前言

本講義介紹知識蒸餾 knowledge distillation（KD）：把一個大模型學到的知識，移轉給一個小模型的訓練方法。 內容依據 Hinton、Vinyals、Dean 於 2015 年發表的論文〈Distilling the Knowledge in a Neural Network〉， 論文的公式、實驗設定與結果都完整列在本講義中，讀者不需要另外查閱原文。

本講義回答三個問題：

1. 為什麼需要把大模型的知識移轉給小模型。
2. 移轉時，小模型要學什麼、損失函數怎麼寫、梯度怎麼算。
3. 這個方法在手寫數字資料上的效果如何。

第 2 至 4 章用同一組自訂的小數字逐步計算，每一步都可以用計算機核對； 第 5 章用 Colab 程式重做論文的 MNIST 實驗。

怎麼讀本講義

章	內容	要動手的地方
1	為什麼要蒸餾	—
2	溫度與軟目標	2.3 節兩張計算表
3	蒸餾的損失函數	3.3 節計算表
4	梯度，以及高溫下的極限	4.2 節計算表
5	MNIST 實驗	Colab 跑一次
6	論文的其他結果	—
7	Colab 程式的使用方式	—
8	練習與解答	三題

術語對照

中文	英文	本講義用的符號或縮寫
知識蒸餾	knowledge distillation	KD
老師模型	teacher model（論文稱 cumbersome model）	—
學生模型	student model（論文稱 distilled model）	—
溫度	temperature	T
軟目標	soft targets	p
硬目標	hard targets	y
移轉集	transfer set	—
交叉熵	cross entropy	C
老師的 logit	logit	v
學生的 logit	logit	z
學生的機率	—	q
正則化	regularization	—

第 1 章　為什麼要蒸餾

1.1 訓練與部署的需求不同

許多昆蟲有幼蟲與成蟲兩種形態：幼蟲適合從環境中攝取養分，成蟲適合移動與繁殖。 論文以此比喻機器學習：訓練階段與部署階段的需求不同，卻常使用同一個模型。

	訓練階段	部署階段
資料量	很大，而且重複性高	一次處理一筆
時間限制	可以計算數天	要即時回應
計算資源	大量 GPU	手機或伺服器
適合的模型	大模型，或多個模型的組合	小模型

訓練時可以使用很大的模型，例如多個模型的組合 ensemble，或是用 Dropout 這類強正則化訓練的單一大模型， 因為這有助於從資料中學到規律。訓練完成後，再用另一種訓練方式，把大模型的知識移轉給適合部署的小模型。 這種訓練方式稱為**蒸餾**。本講義把大模型稱為**老師**，小模型稱為**學生**。

1.2 知識是什麼

一般認為一個模型的知識就是它的參數。依這個看法，模型的架構一改，知識就無法保留。

論文採用另一個看法：模型的知識是它學到的**輸入到輸出的對應**。依這個看法， 只要學生對每一個輸入給出和老師相近的輸出，知識就移轉過去了，兩者的架構不必相同。

1.3 錯誤答案的機率也有資訊

分類模型訓練時，目標是提高正確類別的機率；但訓練完成後，它也會給每個錯誤類別一個機率。 這些機率雖然小，大小卻不相同，反映模型怎麼看待輸入。論文舉了兩個例子：

輸入	被認為是	機率
一張 BMW 的照片	垃圾車	很小
一張 BMW 的照片	紅蘿蔔	比垃圾車小很多倍

輸入	被認為是 3	被認為是 7
第一種寫法的 2	10^{-6}	10^{-9}
第二種寫法的 2	10^{-9}	10^{-6}

第二個例子說明：第一種寫法的 2 比較像 3，第二種寫法的 2 比較像 7。這是資料之間的相似關係，只看正確答案得不到。 大模型通常比小模型更懂得如何類推到新資料，學生如果學到這些機率，也就學到老師類推的方式。

問題在於 10^{-6} 與 10^{-9} 都太接近 0。用一般的交叉熵訓練學生時，這兩個數字幾乎不影響損失，學生學不到它們。 Caruana 等人的做法是讓學生直接學老師 softmax 之前的數值（logit），以平方差作為損失。 本論文提出的做法是提高 softmax 的**溫度**，第 4.3 節會說明，前者是後者的特例。

第 2 章　溫度與軟目標

2.1 溫度軟化的 softmax

神經網路最後一層輸出的數值稱為 logit，第 i 個類別的 logit 記為 z_i。softmax 把 logit 轉成機率 q_i。論文的式 (1) 在 softmax 中加入溫度 T：

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{1}$$

$T = 1$ 就是一般的 softmax。T 越大，各類別的機率越接近，機率分布越平均。

2.2 示範資料

本講義第 2 至 4 章使用同一組自訂的數字。輸入是一張手寫的 2，為了能手算，只取「2、3、7」三個類別。

類別	老師 logit v	學生 logit z	正確答案 y
2	8	4	1
3	4	2	0
7	0	2	0

老師很確定這是 2，也認為 3 比 7 像；學生分得出 2，但對 3 和 7 給出相同的 logit。這組數字是為了手算而自訂的，不是實際模型的輸出。

2.3 逐步計算

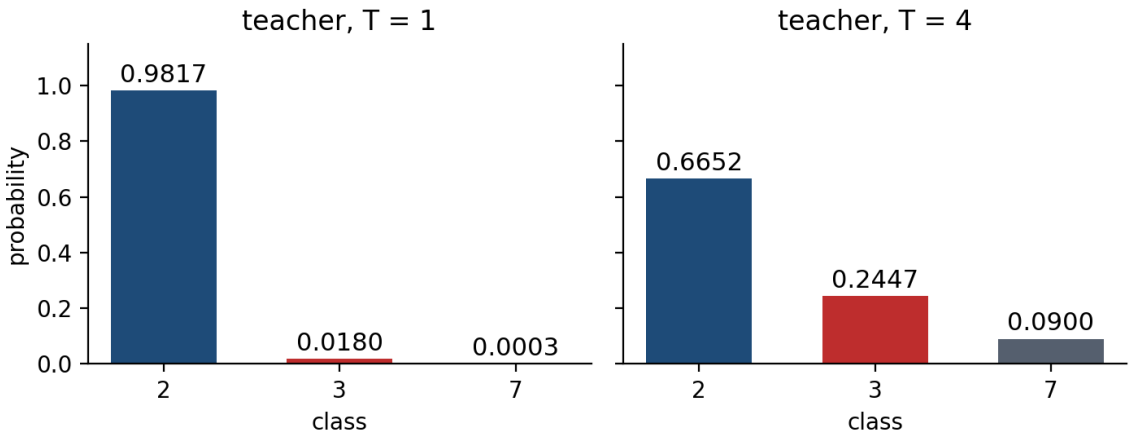
式 (1) 分四步計算：每個 logit 除以 T、取指數、加總、各自除以總和。

$T = 1$

類別	v	第 1 步 v/T	第 2 步 $\exp(v/T)$	第 4 步 $p = \exp \div$ 總和
2	8	8.0000	2980.9580	0.9817
3	4	4.0000	54.5982	0.0180
7	0	0.0000	1.0000	0.0003
第 3 步 總和			3036.5561	

$T = 4$

類別	v	第 1 步 v/T	第 2 步 $\exp(v/T)$	第 4 步 $p = \exp \div$ 總和
2	8	2.0000	7.3891	0.6652
3	4	1.0000	2.7183	0.2447
7	0	0.0000	1.0000	0.0900
第 3 步 總和			11.1073	



老師在兩種溫度下的機率

$T=1$ 時，3 和 7 的機率是 0.0180 與 0.0003，都接近 0；$T=4$ 時變成 0.2447 與 0.0900。 兩者的比值等於 $\exp\big((4-0)/T\big)$：$T=1$ 時是 $e^4=54.60$，$T=4$ 時是 $e^1=2.72$。 提高溫度不改變類別的排序，只縮小機率之間的差距，使錯誤類別的機率大到足以影響訓練。

2.4 軟目標與硬目標

老師在高溫下給出的機率稱為**軟目標**，本例在 $T=4$ 時為 $p=(0.6652, 0.2447, 0.0900)$。 正確答案本身稱為**硬目標**，本例為 $y=(1,0,0)$。

論文指出，軟目標的熵較高時，每一筆訓練資料提供的資訊比硬目標多，各筆資料之間的梯度變異也比較小； 因此學生通常可以用比老師少得多的資料、較大的學習率訓練。

用來訓練學生的資料稱為**移轉集**。移轉集可以是沒有標籤的資料，也可以直接使用原本的訓練集。 論文發現使用原本的訓練集效果很好，尤其是損失函數再加上一小項正確答案時，見第 3 章。

第 3 章　蒸餾的損失函數

3.1 兩個目標函數

學生的損失由兩項加權平均組成：

- **軟目標項**：老師的軟目標 p 與學生的機率 q 的交叉熵。p 與 q 都在**同一個溫度** T 下計算。

$$C_{soft} = -\sum_i p_i \log q_i$$

- **硬目標項**：正確答案與學生在 $T=1$ 下的機率的交叉熵。因為 y 只有正確類別是 1，這一項只剩一個對數：

$$C_{hard} = -\log q^{(T=1)}_{\text{correct}}$$

兩項使用學生的同一組 logit，差別只在溫度。總損失為

$$L = \alpha\, T^2\, C_{soft} + (1-\alpha)\, C_{hard}$$

3.2 兩個係數

- α 是軟目標項的權重。論文指出，硬目標項的權重取得比軟目標項低很多，結果通常最好。本講義取 $\alpha = 0.9$。
- T^2：軟目標項產生的梯度大小與 $1/T^2$ 成正比（第 4.3 節驗證）。乘上 T^2 之後，實驗時改變溫度，兩項的相對比重也大致不變。

學生訓練完成後，使用時溫度回到 $T=1$。

3.3 逐步計算

第一步：學生在 $T = 4$ 的機率 q

類別	z	$z/4$	$\exp(z/4)$	q
2	4	1.0000	2.7183	0.4519
3	2	0.5000	1.6487	0.2741
7	2	0.5000	1.6487	0.2741
總和			6.0157	

第二步：軟目標項 C_{soft}

類別	老師 p	$\log q$	$p \times \log q$
2	0.6652	−0.7944	−0.5285
3	0.2447	−1.2944	−0.3168
7	0.0900	−1.2944	−0.1165
總和			−0.9618

$C_{soft} = -(-0.9618) = 0.9618$。log 是自然對數；各格四捨五入到小數第 4 位，總和以未進位的值計算。

第三步：學生在 $T = 1$ 的機率與硬目標項 C_{hard}

類別	z	$\exp(z)$	$q^{(T=1)}$
2	4	54.5982	0.7870
3	2	7.3891	0.1065
7	2	7.3891	0.1065
總和		69.3763	

正確答案是 2，所以 $C_{hard} = -\log 0.7870 = 0.2395$。

第四步：總損失

項	算式	值
軟目標項	$0.9 \times 4^2 \times 0.9618$	13.8493
硬目標項	0.1×0.2395	0.0240
總損失 L	兩項相加	13.8732

第 4 章　梯度，以及高溫下的極限

4.1 式 (2)

訓練學生時，需要損失對學生 logit z_i 的梯度。軟目標項的梯度是論文的式 (2)：

$$\frac{\partial C_{soft}}{\partial z_i} = \frac{1}{T}\left(q_i - p_i\right) \tag{2}$$

推導如下。$q = \mathrm{softmax}(z/T)$，所以 $\log q_j = z_j/T - \log \sum_k \exp(z_k/T)$，對 z_i 微分得 $\frac{\partial \log q_j}{\partial z_i} = \frac{1}{T}(\delta_{ij} - q_i)$，其中 δ_{ij} 在 $i=j$ 時為 1，否則為 0。代入 C_{soft}：

$$\frac{\partial C_{soft}}{\partial z_i} = -\sum_j p_j \cdot \frac{1}{T}(\delta_{ij} - q_i) = \frac{1}{T}\Big(q_i \sum_j p_j - p_i\Big) = \frac{1}{T}(q_i - p_i)$$

最後一步用到 $\sum_j p_j = 1$。同樣的推導在 $T=1$ 時給出硬目標項的梯度 $q_i^{(T=1)} - y_i$。

4.2 逐步計算

類別	$q - p$	$\div T$（式 2）	$\times \alpha T^2 = 14.4$	$q^{(T=1)} - y$	$\times (1-\alpha) = 0.1$	總梯度
2	−0.2134	−0.0533	−0.7682	−0.2130	−0.0213	−0.7895
3	0.0293	0.0073	0.1056	0.1065	0.0107	0.1163
7	0.1840	0.0460	0.6625	0.1065	0.0107	0.6732

梯度下降法沿著梯度的反方向更新參數：2 的 logit 會變大，7 的 logit 下降最多，3 只小幅下降。 學生因此往老師的看法「2 最像、3 其次、7 最不像」靠近。只用硬目標項時，3 與 7 的梯度相同（都是 0.1065），學生學不到兩者的差別。

Colab 程式第 3 段用 TensorFlow 的自動微分 `GradientTape` 重算同一組數字，得到相同的損失 13.8732 與相同的總梯度。

4.3 為什麼乘 T^2

同一組 logit，在三種溫度下計算式 (2)：

T	式 (2) 梯度的最大絕對值	乘 T^2 後
1	0.194704	0.1947
4	0.053345	0.8535
20	0.002637	1.0547

T 從 4 升到 20，梯度縮小約 20 倍，接近 $(20/4)^2 = 25$ 倍；乘上 T^2 之後是 0.85 與 1.05，大小相近。$1/T^2$ 的比例是下一節高溫近似的結果，$T = 1$ 不在這個近似的範圍內。

4.4 高溫下等於比對 logit

當 x 很小時，$\exp(x) \approx 1 + x$。若溫度遠大於 logit，z_i/T 與 v_i/T 都很小，式 (2) 可近似為論文的式 (3)：

$$\frac{\partial C_{soft}}{\partial z_i} \approx \frac{1}{T}\left(\frac{1 + z_i/T}{N + \sum_j z_j/T} - \frac{1 + v_i/T}{N + \sum_j v_j/T}\right) \tag{3}$$

N 是類別數。再假設老師與學生的 logit 各自減去平均值，即 $\sum_j z_j = \sum_j v_j = 0$，式 (3) 化簡為式 (4)：

$$\frac{\partial C_{soft}}{\partial z_i} \approx \frac{1}{NT^2}\left(z_i - v_i\right) \tag{4}$$

這正是最小化 $\frac{1}{2}(z_i - v_i)^2$ 的梯度乘上常數 $\frac{1}{NT^2}$。也就是說，**高溫時的蒸餾，等於讓學生的 logit 直接貼齊老師的 logit**。式 (4) 也說明了第 4.3 節的 $1/T^2$。

以本例驗證：減去平均後 $z = (1.3333, -0.6667, -0.6667)$，$v = (4, 0, -4)$，$N = 3$，$T = 20$。

類別	式 (2) 精確值	式 (4) 近似值
2	−0.00229	−0.00222
3	−0.00034	−0.00056
7	0.00264	0.00278

溫度較低時，式 (2) 對「比平均值負很多的 logit」幾乎不要求貼齊。這可能是好事：這些 logit 在老師訓練時幾乎不受損失函數約束，可能含有很多雜訊； 也可能是壞事：它們也可能帶有老師的知識。論文的實驗發現，學生太小、學不下老師全部的知識時，中間的溫度最好，表示忽略這些很負的 logit 是有幫助的（第 5.4 節）。

第 5 章　MNIST 實驗

5.1 論文的設定與結果

MNIST 是 28×28 灰階的手寫數字影像，訓練集 60,000 張，測試集 10,000 張。論文第 3 節的設定：

	架構	正則化	測試集答錯張數
老師	兩個隱藏層，各 1,200 個 ReLU 神經元	Dropout、權重限制，輸入影像隨機平移最多 2 個像素	67
學生，只學正確答案	兩個隱藏層，各 800 個 ReLU 神經元	無	146
學生，蒸餾 $T = 20$	同上	只有蒸餾的軟目標	74

蒸餾使學生的答錯張數從 146 降到 74，接近老師的 67。 老師的訓練資料經過平移，學生的移轉集沒有平移；學生卻學到了老師處理平移影像的方式，因為這項資訊包含在軟目標中。

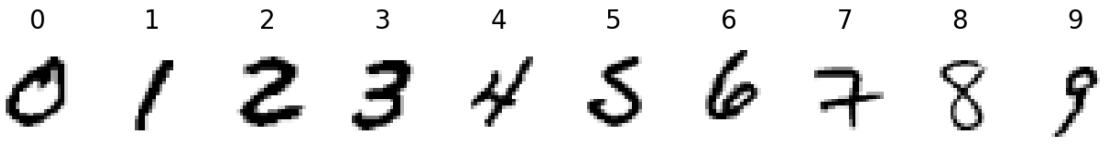
論文另外報告兩項結果：

1. 學生每層有 300 個以上神經元時，8 以上的溫度結果相近；學生每層縮小到 30 個時，2.5 到 4 的溫度明顯比更高或更低的溫度好。
2. 移轉集拿掉所有的 3，學生從沒看過 3。蒸餾的學生仍只答錯 206 張，其中 133 張是測試集 1,010 張 3 中的錯誤。 答錯的 3 大多是因為 3 這一類的偏差值學得太低；把 3 的偏差值加 3.5（這個值以測試集挑選），答錯降為 109 張，其中 3 只錯 14 張，3 的答對率 98.6%。 若移轉集只保留 7 和 8，學生的測試錯誤率為 47.3%；把 7 和 8 的偏差值減 7.6 後降為 13.2%。

5.2 本講義的重做

本講義的 Colab 程式重做上述三項結果。為了在 Colab 免費版數分鐘內完成，資料與學生都縮小：

	論文	本講義
訓練資料	60,000 張	10,000 張，0 到 9 各 1,000 張
測試資料	10,000 張	同一個測試集 10,000 張，其中 3 有 1,010 張
老師	1200-1200，Dropout，平移 2 像素	1200-1200，Dropout，平移 2 像素
學生	800-800；另試 30-30	100-100；另試 30-30
溫度	20，另試多種	1、2、4、8、20
α	硬目標權重取低	0.9



MNIST 訓練影像

以下數字是 2026 年 10 月 19 日在本機 CPU 執行 Colab 程式的結果（Python 3.13、TensorFlow 2.21、Keras 3.13）。 在 Colab 上重跑，數字會有小幅差異。

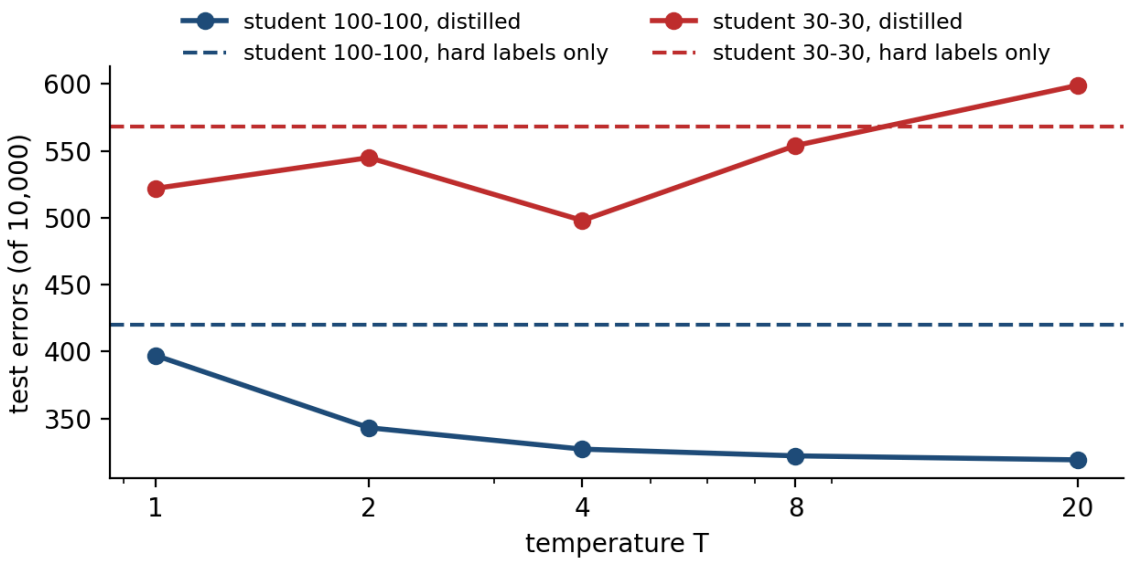
5.3 結果一：蒸餾的學生答錯較少

模型	測試集答錯張數
老師 1200-1200	222
學生 100-100，只學正確答案	420
學生 100-100，蒸餾 $T = 20$	319

學生與老師之間的差距，由 198 張縮小為 97 張。

5.4 結果二：學生很小時，溫度取中間值

T	學生 100-100	學生 30-30
只學正確答案	420	568
1	397	522
2	343	545
4	327	498
8	322	554
20	319	599



答錯張數與溫度

學生 100-100 在所有溫度下都比只學正確答案少錯，溫度越高越好。學生 30-30 在 $T = 4$ 最好，$T = 20$ 反而比只學正確答案多錯。 這與論文的觀察一致：溫度越高，越接近第 4.4 節的比對 logit，

連非常負的 logit 也要貼齊，容量太小的學生學不來。

5.5 結果三：學生從沒看過 3

移轉集拿掉所有的 3，剩 9,000 張。學生 100-100。

模型	總共答錯	其中 3 答錯	3 的答對率
只學正確答案	1372	1010 / 1010	0.0%
蒸餾 $T = 8$	727	457 / 1010	54.8%
蒸餾 $T = 8$，3 的 logit 加 3.0	442	104 / 1010	89.7%

只學正確答案的學生，訓練時沒有任何一張的答案是 3，不可能答出 3。 蒸餾的學生雖然沒看過 3，老師對其他數字給出的軟目標中，仍包含「這張有幾分像 3」的資訊，因此學生認得出一半以上的 3； 加上偏差值修正後，答對率接近九成。加多少是用測試集挑選的，與論文相同，所以這個數字偏樂觀。

第 6 章　論文的其他結果

本章的數字取自論文，本講義沒有重做。

6.1 語音辨識

論文以 Android 語音搜尋所用的聲學模型（8 個隱藏層、每層 2,560 個神經元、輸出 14,000 個類別、約 8,500 萬個參數）為基準， 獨立訓練 10 個相同架構、不同初始值的模型，再把 10 個模型的組合蒸餾成一個同樣大小的模型。

系統	測試框架準確率	詞錯誤率 WER
單一模型	58.9%	10.9%
10 個模型的組合	61.1%	10.7%
蒸餾的單一模型	60.8%	10.7%

10 個模型的組合帶來的進步，超過 80% 移轉到了單一模型。

6.2 軟目標可以當作正則化

同一個語音模型只用 3% 的訓練資料（約 2,000 萬筆）訓練：

訓練方式	訓練資料	訓練框架準確率	測試框架準確率
硬目標	100%	63.4%	58.9%
硬目標	3%	67.3%	44.5%
軟目標	3%	65.4%	57.0%

只用硬目標時嚴重過擬合，必須提前停止；改用全部資料訓練出的模型所提供的軟目標，3% 的資料就能接近全部資料的結果，而且不需要提前停止。

6.3 專家模型

在 1 億張影像、15,000 個類別的 JFT 資料集上，論文另訓練 61 個專家模型 specialist models。 每個專家負責一群容易混淆的類別（例如多種橋、多款車），其餘類別併成一個「其他」類。 專家以原模型的權重初始化，可以各自獨立、平行訓練，數天內完成。 與原模型合用時，測試準確率由 25.0% 提高到 26.1%；只看專家負責的類別，由 43.1% 提高到 45.9%。

第 7 章　Colab 程式的使用方式

程式 `knowledge_distillation.ipynb` 在 Colab 開啟後，由上往下逐格執行即可。共八段：

段	內容	對應本講義
1	載入資料	5.2 節
2	溫度軟化的 softmax 手算例	2.3 節
3	損失與梯度手算例，並以自動微分核對	3.3、4.2、4.3、4.4 節
4	訓練老師	5.2 節
5	學生 100-100 的溫度比較	5.3、5.4 節
6	學生 30-30 的溫度比較	5.4 節
7	學生從沒看過 3	5.5 節
8	結果整理	—

資料檔 `kd_mnist.npz`（約 3.3 MB）有兩種取得方式，程式會自動判斷：

1. 把老師提供的 `kd_mnist.npz` 拖到 Colab 左側的「檔案」區，放在預設資料夾 `/content`。
2. 左側沒有這個檔時，程式從課程的 GitHub repo 下載同一個檔。

在 CPU 上全部執行約需數分鐘；改用 GPU（「執行階段 → 變更執行階段類型 → T4 GPU」）會更快。

第 8 章　練習與解答

練習一　$T = 2$ 時，老師 $v = (8, 4, 0)$ 的軟目標是多少？依第 2.3 節的四步計算。

練習二　學生 z 改成 $(8, 4, 0)$，和老師相同，$T = 4$。式 (2) 的梯度是多少？為什麼？

練習三　在 Colab 第 7 段改成拿掉數字 7，重做一次，記錄 7 的答對率，以及加偏差值後的答對率。

解答一　$v/2 = (4, 2, 0)$；取指數得 $(54.5982, 7.3891, 1.0000)$；總和 62.9873； 軟目標 $p = (0.8668, 0.1173, 0.0159)$。比 $T = 1$ 平均，比 $T = 4$ 集中。

解答二　學生與老師的 logit 相同，所以在同一溫度下 $q = p$，式 (2) 的 $q_i - p_i$ 全為 0，軟目標項的梯度為 $(0, 0, 0)$：學生已經完全貼齊老師，軟目標項不再推動它。 總梯度只剩硬目標項 $0.1 \times (q^{(T=1)} - y) = 0.1 \times (0.9817 - 1,\ 0.0180,\ 0.0003) = (-0.0018, 0.0018, 0.0000)$。

解答三　結果依每次訓練而不同，沒有固定答案。記錄時同時寫下總共答錯張數、7 答錯張數，以及加了多少偏差值。

參考文獻

1. G. Hinton, O. Vinyals, J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv:1503.02531, 2015.
2. C. Buciluǎ, R. Caruana, A. Niculescu-Mizil, “Model compression,” in *Proc. 12th ACM SIGKDD*, 2006, pp. 535–541.
3. Y. LeCun, C. Cortes, C. J. C. Burges, The MNIST database of handwritten digits.